Informatik 09 - Objektorientierte Programmierung mit Java

Informatik 09 - Objektorientierte Programmierung mit Java

Programmieren: Sprachen

Karol-Java Übersetzung

Übersicht Befehlsarten

Semikolon und geschweifte Klammern

Strukturierung von Programmcode - Syntax

Strukturierung von Programmcode - Semantik: Methoden

Übersetzungstraining: Methoden

Karol mit Java und Struktogramme

Programmablauf grafisch darstellen: Struktogramme

Struktogramm-Training

Taschenrechner

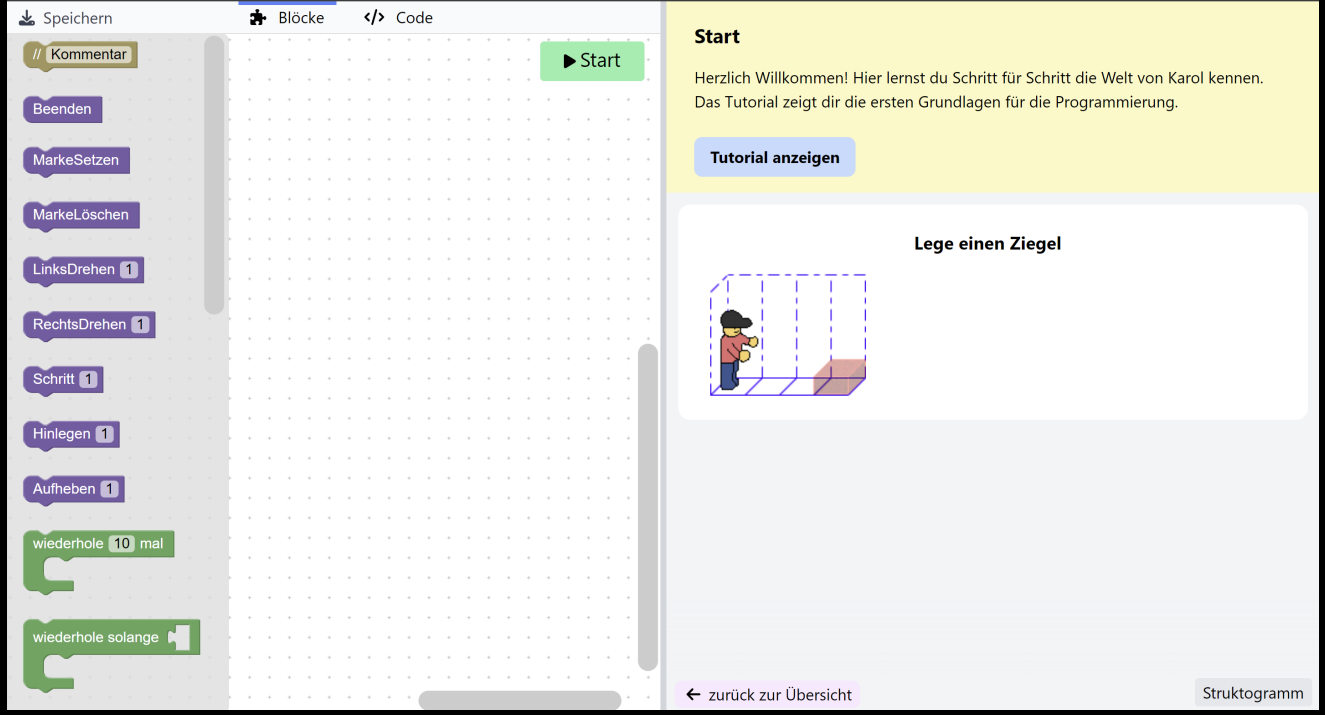
Methodenköpfe

Methoden im Klassendiagramm

Attribute im Klassendiagramm

Attribute im Klassendiagramm

Struktogramme und Klassendiagramme


Speichern
Blöcke
Code
Kommentar
Beenden
MarkeSetzen
MarkeLöschen
LinksDrehen 1
RechtsDrehen 1
Schritt 1
Hinlegen 1
Aufheben 1
wiederhole 10 mal
wiederhole solange
Start
Start
Herzlich Willkommen! Hier lernst du Schritt für Schritt die Welt von Karol kennen.
Das Tutorial zeigt dir die ersten Grundlagen für die Programmierung.
Tutorial anzeigen
Lege einen Ziegel
zurück zur Übersicht
Struktogramm

Programmieren: Sprachen

Beim Programmieren schreibt man präzise und unmissverständliche Befehle, die dann vom in Maschinensprache übersetzt werden.

Hierfür gibt es verschiedene Programmiersprachen mit jeweils einer festgelegte **Grammatik und Rechtschreibung**). Dem gegenüber steht die **Logik eines Programms (=**), die weitgehend unabhängig von der Programmiersprache ist.

Bei den einsteigerfreundlichen **block-basierten Sprachen wie Blockly oder Online-Karol** übernehmen die Blöcke und ihre Verbindungsmöglichkeiten weitgehend die Einhaltung des Syntax. Bei **text-basierten Sprachen wie** **oder** müssen die Entwickler:innen selbst auf deren Einhaltung achten. Diese Sprachen

bieten dadurch aber auch weitaus mehr Möglichkeiten (**man sagt auch: sie sind mächtiger**).
Dieses Skript arbeitet parallel mit den blockbasierten Sprachen Blockly und Online-Karol und der weit verbreiteten text-basierten und **objektorientierten** Sprache **Java**.

Programmieren: Sprachen

Beim Programmieren schreibt man präzise und unmissverständliche Befehle, die dann vom `Compiler` in Maschinensprache übersetzt werden.
Hierfür gibt es verschiedene Programmiersprachen mit jeweils einer festgelegte **Grammatik und Rechtschreibung (=)**. Dem gegenüber steht die **Logik eines Programms (=)**, die weitgehend unabhängig von der Programmiersprache ist.

Bei den einsteigerfreundlichen **block-basierten Sprachen wie Blockly oder Online-Karol** übernehmen die Blöcke und ihre Verbindungsmöglichkeiten weitgehend die Einhaltung des Syntax. Bei **text-basierten Sprachen wie oder** müssen die Entwickler:innen selbst auf deren Einhaltung achten. Diese Sprachen bieten dadurch aber auch weitaus mehr Möglichkeiten (**man sagt auch: sie sind mächtiger**).
Dieses Skript arbeitet parallel mit den blockbasierten Sprachen Blockly und Online-Karol und der weit verbreiteten text-basierten und **objektorientierten** Sprache **Java**.

Programmieren: Sprachen

Beim Programmieren schreibt man präzise und unmissverständliche Befehle, die dann vom `Compiler` in Maschinensprache übersetzt werden.
Hierfür gibt es verschiedene Programmiersprachen mit jeweils einer festgelegte **Grammatik und Rechtschreibung (= `Syntax`)**. Dem gegenüber steht die **Logik eines Programms (=)**, die weitgehend unabhängig von der Programmiersprache ist.

Bei den einsteigerfreundlichen **block-basierten Sprachen wie Blockly oder Online-Karol** übernehmen die Blöcke und ihre Verbindungsmöglichkeiten weitgehend die Einhaltung des Syntax. Bei **text-basierten Sprachen wie oder** müssen die Entwickler:innen selbst auf deren Einhaltung achten. Diese Sprachen bieten dadurch aber auch weitaus mehr Möglichkeiten (**man sagt auch: sie sind mächtiger**).
Dieses Skript arbeitet parallel mit den blockbasierten Sprachen Blockly und Online-Karol und der weit verbreiteten text-basierten und **objektorientierten** Sprache **Java**.

Programmieren: Sprachen

Beim Programmieren schreibt man präzise und unmissverständliche Befehle, die dann vom `Compiler` in Maschinensprache übersetzt werden.
Hierfür gibt es verschiedene Programmiersprachen mit jeweils einer festgelegte **Grammatik und Rechtschreibung (= `Syntax`)**. Dem gegenüber steht die **Logik eines Programms (= `Semantik`)**, die weitgehend unabhängig von der Programmiersprache ist.

Bei den einsteigerfreundlichen **block-basierten Sprachen wie Blockly oder Online-Karol** übernehmen die Blöcke und ihre Verbindungsmöglichkeiten weitgehend die Einhaltung des Syntax. Bei **text-basierten Sprachen wie oder** müssen die Entwickler:innen selbst auf deren Einhaltung achten. Diese Sprachen bieten dadurch aber auch weitaus mehr Möglichkeiten (**man sagt auch: sie sind mächtiger**).
Dieses Skript arbeitet parallel mit den blockbasierten Sprachen Blockly und Online-Karol und der weit verbreiteten text-basierten und **objektorientierten** Sprache **Java**.

Programmieren: Sprachen

Beim Programmieren schreibt man präzise und unmissverständliche Befehle, die dann vom `Compiler` in Maschinensprache übersetzt werden.
Hierfür gibt es verschiedene Programmiersprachen mit jeweils einer festgelegte **Grammatik und Rechtschreibung (= `Syntax`)**. Dem gegenüber steht die **Logik eines Programms (= `Semantik`)**, die weitgehend unabhängig von der Programmiersprache ist.

Bei den einsteigerfreundlichen **block-basierten Sprachen wie Blockly oder Online-Karol** übernehmen die Blöcke und ihre Verbindungsmöglichkeiten weitgehend die Einhaltung des Syntax. Bei **text-basierten Sprachen wie `Java` oder** müssen die Entwickler:innen selbst auf deren Einhaltung achten. Diese Sprachen bieten dadurch aber auch weitaus mehr Möglichkeiten (**man sagt auch: sie sind mächtiger**).
Dieses Skript arbeitet parallel mit den blockbasierten Sprachen Blockly und Online-Karol und der weit verbreiteten text-basierten und **objektorientierten** Sprache **Java**.

Programmieren: Sprachen

Beim Programmieren schreibt man präzise und unmissverständliche Befehle, die dann vom `Compiler` in Maschinensprache übersetzt werden.
Hierfür gibt es verschiedene Programmiersprachen mit jeweils einer festgelegte **Grammatik und Rechtschreibung (= `Syntax`)**. Dem gegenüber steht die **Logik eines Programms (= `Semantik`)**, die weitgehend unabhängig von der Programmiersprache ist.

Bei den einsteigerfreundlichen **block-basierten Sprachen wie Blockly oder Online-Karol** übernehmen die Blöcke und ihre Verbindungsmöglichkeiten weitgehend die Einhaltung des Syntax. Bei **text-basierten Sprachen wie `Java` oder `Python`** müssen die Entwickler:innen selbst auf deren Einhaltung achten. Diese Sprachen bieten dadurch aber auch weitaus mehr Möglichkeiten (**man sagt auch: sie sind mächtiger**).
Dieses Skript arbeitet parallel mit den blockbasierten Sprachen Blockly und Online-Karol und der weit verbreiteten text-basierten und **objektorientierten** Sprache **Java**.

Karol-Java Übersetzung

1. Bearbeite die Aufgaben von Robot Karol online: `karol.arrrg.de`
2. Schalte in der Code-Ansicht die Sprache auf Java um und notiere auf dem AB jeweils zu welchem Java-Code der Block übersetzt wird. Achte hierbei auf Einrückungen!

Karol-Java Übersetzung

1. Bearbeite die Aufgaben von Robot Karol online: `karol.arrrg.de`
2. Schalte in der Code-Ansicht die Sprache auf Java um und notiere auf dem AB jeweils zu welchem Java-Code der Block übersetzt wird. Achte hierbei auf Einrückungen!

```
karol.schritt(2);
```

Karol-Java Übersetzung

1. Bearbeite die Aufgaben von Robot Karol online: `karol.arrrg.de`
2. Schalte in der Code-Ansicht die Sprache auf Java um und notiere auf dem AB jeweils zu welchem Java-Code der Block übersetzt wird. Achte hierbei auf Einrückungen!

```
karol.schritt(2);
```

```
karol.markeSetzen();
```

Karol-Java Übersetzung

1. Bearbeite die Aufgaben von Robot Karol online: `karol.arrrg.de`
2. Schalte in der Code-Ansicht die Sprache auf Java um und notiere auf dem AB jeweils zu welchem Java-Code der Block übersetzt wird. Achte hierbei auf Einrückungen!

```
karol.schritt(2);
```

```
karol.markeSetzen();
```

```
karol.schritt(10);
```

Karol-Java Übersetzung

1. Bearbeite die Aufgaben von Robot Karol online: `karol.arrrg.de`
2. Schalte in der Code-Ansicht die Sprache auf Java um und notiere auf dem AB jeweils zu welchem Java-Code der Block übersetzt wird. Achte hierbei auf Einrückungen!

```
karol.schritt(2);
```

```
karol.markeSetzen();
```

```
karol.schritt(10);
```

```
while(true) {
      karol.linksDrehen(1);
}
```

Karol-Java Übersetzung

1. Bearbeite die Aufgaben von Robot Karol online: `karol.arrrg.de`
2. Schalte in der Code-Ansicht die Sprache auf Java um und notiere auf dem AB jeweils zu welchem Java-Code der Block übersetzt wird. Achte hierbei auf Einrückungen!

```
karol.schritt(2);
```

```
karol.markeSetzen();
```

```
karol.schritt(10);
```

```
while(true) {
      karol.linksDrehen(1);
}
```

Karol-Java Übersetzung

1. Bearbeite die Aufgaben von Robot Karol online: `karol.arrrg.de`
2. Schalte in der Code-Ansicht die Sprache auf Java um und notiere auf dem AB jeweils zu welchem Java-Code der Block übersetzt wird. Achte hierbei auf Einrückungen!

```
karol.schritt(2);
```

```
karol.markeSetzen();
```

```
karol.schritt(10);
```

```
while(true) {
      karol.linksDrehen(1);
}
```

Karol-Java Übersetzung

1. Bearbeite die Aufgaben von Robot Karol online: `karol.arrrg.de`
2. Schalte in der Code-Ansicht die Sprache auf Java um und notiere auf dem AB jeweils zu welchem Java-Code der Block übersetzt wird. Achte hierbei auf Einrückungen!

```
karol.schritt(2);
```

```
karol.markeSetzen();
```

```
karol.schritt(10);
```

```
while(true) {
      karol.linksDrehen(1);
}
```

Übersicht Befehlsarten

Es gibt viele verschiedene Befehlsarten. Die für uns wichtigsten sind:

Übersicht Befehlsarten

Es gibt viele verschiedene Befehlsarten. Die für uns wichtigsten sind:

Übersicht Befehlsarten

Es gibt viele verschiedene Befehlsarten. Die für uns wichtigsten sind:

Übersicht Befehlsarten

Es gibt viele verschiedene Befehlsarten. Die für uns wichtigsten sind:

Übersicht Befehlsarten

Es gibt viele verschiedene Befehlsarten. Die für uns wichtigsten sind:

Übersicht Befehlsarten

Es gibt viele verschiedene Befehlsarten. Die für uns wichtigsten sind:

Übersicht Befehlsarten

Es gibt viele verschiedene Befehlsarten. Die für uns wichtigsten sind:

Übersicht Befehlsarten

Es gibt viele verschiedene Befehlsarten. Die für uns wichtigsten sind:

Übersicht Befehlsarten

Es gibt viele verschiedene Befehlsarten. Die für uns wichtigsten sind:

Übersicht Befehlsarten

Es gibt viele verschiedene Befehlsarten. Die für uns wichtigsten sind:

Übersicht Befehlsarten

Es gibt viele verschiedene Befehlsarten. Die für uns wichtigsten sind:

Übersicht Befehlsarten

Es gibt viele verschiedene Befehlsarten. Die für uns wichtigsten sind:

Übersicht Befehlsarten

Es gibt viele verschiedene Befehlsarten. Die für uns wichtigsten sind:

Übersicht Befehlsarten

Es gibt viele verschiedene Befehlsarten. Die für uns wichtigsten sind:

Übersicht Befehlsarten

Es gibt viele verschiedene Befehlsarten. Die für uns wichtigsten sind:

Übersicht Befehlsarten

Es gibt viele verschiedene Befehlsarten. Die für uns wichtigsten sind:

Übersicht Befehlsarten

Es gibt viele verschiedene Befehlsarten. Die für uns wichtigsten sind:

Übersicht Befehlsarten

Es gibt viele verschiedene Befehlsarten. Die für uns wichtigsten sind:

Übersicht Befehlsarten

Es gibt viele verschiedene Befehlsarten. Die für uns wichtigsten sind:

Übersicht Befehlsarten

Es gibt viele verschiedene Befehlsarten. Die für uns wichtigsten sind:

Übersicht Befehlsarten

Es gibt viele verschiedene Befehlsarten. Die für uns wichtigsten sind:

Übersicht Befehlsarten

Es gibt viele verschiedene Befehlsarten. Die für uns wichtigsten sind:

Übersicht Befehlsarten

Es gibt viele verschiedene Befehlsarten. Die für uns wichtigsten sind:

Übersicht Befehlsarten

Es gibt viele verschiedene Befehlsarten. Die für uns wichtigsten sind:

Übersicht Befehlsarten

Es gibt viele verschiedene Befehlsarten. Die für uns wichtigsten sind:

Übersicht Befehlsarten

Es gibt viele verschiedene Befehlsarten. Die für uns wichtigsten sind:

Übersicht Befehlsarten

Es gibt viele verschiedene Befehlsarten. Die für uns wichtigsten sind:

Übersicht Befehlsarten

Es gibt viele verschiedene Befehlsarten. Die für uns wichtigsten sind:

Semikolon und geschweifte Klammern

Wann macht man in Java ein Semikolon und wann setzt man geschweifte Klammern ein?

1. Überlege zuerst allein (2 Minuten)
2. Besprecht eure Überlegungen zu zweit und probiert sie in Online-Karol zur bestätigen (4 Minuten)

Strukturierung von Programmcode - Syntax

In Java wird Programmcode vor allem mit strukturiert:

markieren das Ende eines **einfachen Befehls oder Methodenaufrufs** (normalerweise am Ende der Zeile).

markieren **Code-Abschnitte**, die wiederholt (Schleifen) oder bedingt (Bedingung) ausgeführt werden. Außerdem markieren sie den Methodenrumpf und Anfang und Ende einer Klasse. Je nachdem, was durch die Klammern markiert wird, spricht man synonym zu der Klammern auch von der Klasse, Methode, Schleife, etc.

Strukturierung von Programmcode - Syntax

In Java wird Programmcode vor allem mit **`Semikolons und geschweiften Klammern`** strukturiert:

markieren das Ende eines **einfachen Befehls oder Methodenaufrufs** (normalerweise am Ende der Zeile).

markieren **Code-Abschnitte**, die wiederholt (Schleifen) oder bedingt (Bedingung) ausgeführt werden. Außerdem markieren sie den Methodenrumpf und Anfang und Ende einer Klasse. Je nachdem, was durch die Klammern markiert wird, spricht man synonym zu der Klammern auch von der Klasse, Methode, Schleife, etc.

Strukturierung von Programmcode - Syntax

In Java wird Programmcode vor allem mit **`Semikolons und geschweiften Klammern`** strukturiert:

- `Semikolons` markieren das Ende eines **einfachen Befehls oder Methodenaufrufs** (normalerweise am Ende der Zeile).

  markieren **Code-Abschnitte**, die wiederholt (Schleifen) oder bedingt (Bedingung) ausgeführt werden. Außerdem markieren sie den Methodenrumpf und Anfang und Ende einer Klasse. Je nachdem, was durch die Klammern markiert wird, spricht man synonym zu der Klammern auch von der Klasse, Methode, Schleife, etc.

Strukturierung von Programmcode - Syntax

In Java wird Programmcode vor allem mit **`Semikolons und geschweiften Klammern`** strukturiert:

- `Semikolons` markieren das Ende eines **einfachen Befehls oder Methodenaufrufs** (normalerweise am Ende der Zeile).
- `Geschweifte Klammern` markieren **Code-Abschnitte**, die wiederholt (Schleifen) oder bedingt (Bedingung) ausgeführt werden. Außerdem markieren sie den Methodenrumpf und Anfang und Ende einer Klasse. Je nachdem, was durch die Klammern markiert wird, spricht man synonym zu der Klammern auch von der Klasse, Methode, Schleife, etc.

Strukturierung von Programmcode - Syntax

In Java wird Programmcode vor allem mit **`Semikolons und geschweiften Klammern`** strukturiert:

- `Semikolons` markieren das Ende eines **einfachen Befehls oder Methodenaufrufs** (normalerweise am Ende der Zeile).
- `Geschweifte Klammern` markieren **Code-Abschnitte**, die wiederholt (Schleifen) oder bedingt (Bedingung) ausgeführt werden. Außerdem markieren sie den Methodenrumpf und Anfang und Ende einer Klasse. Je nachdem, was durch die Klammern markiert wird, spricht man synonym zu `innerhalb` der Klammern auch von der Klasse, Methode, Schleife, etc.

Strukturierung von Programmcode - Syntax

In Java wird Programmcode vor allem mit **`Semikolons und geschweiften Klammern`** strukturiert:

- `Semikolons` markieren das Ende eines **einfachen Befehls oder Methodenaufrufs** (normalerweise am Ende der Zeile).
- `Geschweifte Klammern` markieren **Code-Abschnitte**, die wiederholt (Schleifen) oder bedingt (Bedingung) ausgeführt werden. Außerdem markieren sie den Methodenrumpf und Anfang und Ende einer Klasse. Je nachdem, was durch die Klammern markiert wird, spricht man synonym zu `innerhalb` der Klammern auch von `innerhalb` der Klasse, Methode, Schleife, etc.

Strukturierung von Programmcode - Semantik: Methoden

Beim Programmieren können eigene Anweisungen/ Befehle/ Methoden/ Funktionen **deklariert** werden. Bei der **Deklaration beschreibt** zunächst mit dem , welche Eigenschaften die Methode hat und anschließend im , wie genau sie sich verhalten soll. Eine deklarierte Methode kann anschließend beliebig oft verwendet (=**aufgerufen**) werden.
Hierdurch kann man sehr lange Programme in übersichtlichere Einzelteile zerlegen, sodass reduziert wird, der Code durch aussagekräftige besser verstanden werden kann und Fehler schneller gefunden werden.
In Java darf außerdem kein auszuführender Programmcode außerhalb von Methoden sein.
Die Deklaration besteht aus **Methodenkopf** und **Methodenrumpf** (von geschweiften Klammern umschlossen):

```
void tueEtwas() {

    karol.schritt(2);
}
```

Der **Aufruf** funktioniert wie ein einfacher Befehl:

```
tueEtwas();
```

Strukturierung von Programmcode - Semantik: Methoden

Beim Programmieren können eigene Anweisungen/ Befehle/ Methoden/ Funktionen **deklariert** werden. Bei der **Deklaration beschreibt** zunächst mit dem `Methodenkopf`, welche Eigenschaften die Methode hat und anschließend im , wie genau sie sich verhalten soll. Eine deklarierte Methode kann anschließend beliebig oft verwendet (=**aufgerufen**) werden.
Hierdurch kann man sehr lange Programme in übersichtlichere Einzelteile zerlegen, sodass reduziert wird, der Code durch aussagekräftige besser verstanden werden kann und Fehler schneller gefunden werden.
In Java darf außerdem kein auszuführender Programmcode außerhalb von Methoden sein.
Die Deklaration besteht aus **Methodenkopf** und **Methodenrumpf** (von geschweiften Klammern umschlossen):

```
void tueEtwas() {

    karol.schritt(2);
}
```

Der **Aufruf** funktioniert wie ein einfacher Befehl:

```
tueEtwas();
```

Strukturierung von Programmcode - Semantik: Methoden

Beim Programmieren können eigene Anweisungen/ Befehle/ Methoden/ Funktionen **deklariert** werden. Bei der **Deklaration beschreibt** zunächst mit dem `Methodenkopf`, welche Eigenschaften die Methode hat und anschließend im `Methodenrumpf`, wie genau sie sich verhalten soll. Eine deklarierte Methode kann anschließend beliebig oft verwendet (=**aufgerufen**) werden.
Hierdurch kann man sehr lange Programme in übersichtlichere Einzelteile zerlegen, sodass reduziert wird, der Code durch aussagekräftige besser verstanden werden kann und Fehler schneller gefunden werden.
In Java darf außerdem kein auszuführender Programmcode außerhalb von Methoden sein.
Die Deklaration besteht aus **Methodenkopf** und **Methodenrumpf** (von geschweiften Klammern umschlossen):

```
void tueEtwas() {

    karol.schritt(2);
}
```

Der **Aufruf** funktioniert wie ein einfacher Befehl:

```
tueEtwas();
```

Strukturierung von Programmcode - Semantik: Methoden

Beim Programmieren können eigene Anweisungen/ Befehle/ Methoden/ Funktionen **deklariert** werden. Bei der **Deklaration beschreibt** zunächst mit dem `Methodenkopf`, welche Eigenschaften die Methode hat und anschließend im `Methodenrumpf`, wie genau sie sich verhalten soll. Eine deklarierte Methode kann anschließend beliebig oft verwendet (=**aufgerufen**) werden.
Hierdurch kann man sehr lange Programme in übersichtlichere Einzelteile zerlegen, sodass `doppelter Code` reduziert wird, der Code durch aussagekräftige besser verstanden werden kann und Fehler schneller gefunden werden.
In Java darf außerdem kein auszuführender Programmcode außerhalb von Methoden sein.
Die Deklaration besteht aus **Methodenkopf** und **Methodenrumpf** (von geschweiften Klammern umschlossen):

```
void tueEtwas() {

    karol.schritt(2);
}
```

Der **Aufruf** funktioniert wie ein einfacher Befehl:

```
tueEtwas();
```

Strukturierung von Programmcode - Semantik: Methoden

Beim Programmieren können eigene Anweisungen/ Befehle/ Methoden/ Funktionen **deklariert** werden. Bei der **Deklaration beschreibt** zunächst mit dem `Methodenkopf`, welche Eigenschaften die Methode hat und anschließend im `Methodenrumpf`, wie genau sie sich verhalten soll. Eine deklarierte Methode kann anschließend beliebig oft verwendet (=**aufgerufen**) werden.
Hierdurch kann man sehr lange Programme in übersichtlichere Einzelteile zerlegen, sodass `doppelter Code` reduziert wird, der Code durch aussagekräftige `Methodennamen` besser verstanden werden kann und Fehler schneller gefunden werden.
In Java darf außerdem kein auszuführender Programmcode außerhalb von Methoden sein.
Die Deklaration besteht aus **Methodenkopf** und **Methodenrumpf** (von geschweiften Klammern umschlossen):

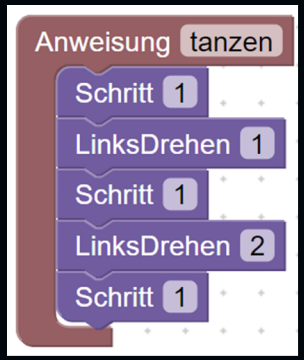
```
void tueEtwas() {

    karol.schritt(2);
}
```

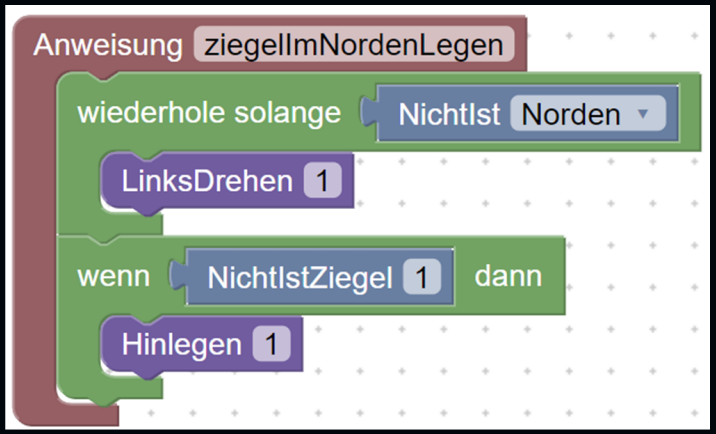
Der **Aufruf** funktioniert wie ein einfacher Befehl:

```
tueEtwas();
```

Übersetzungstraining: Methoden

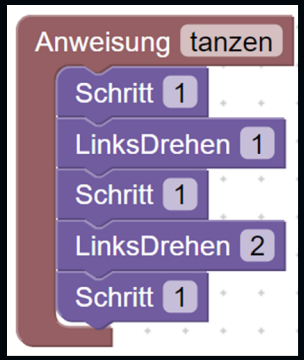
Übersetze die folgenden Code-Schnippsel (**code snippets**) in Java-Code ohne den Computer zu verwenden.

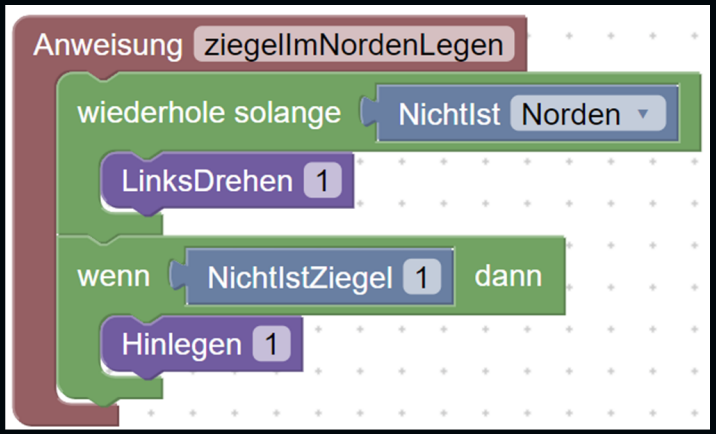




Übersetzungstraining: Methoden

Übersetze die folgenden Code-Schnippsel (**code snippets**) in Java-Code ohne den Computer zu verwenden.

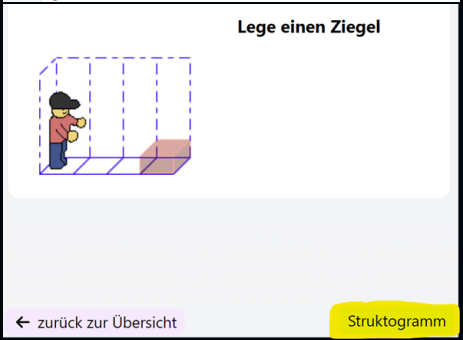




Karol mit Java und Struktogramme

- Bearbeite die Robot Karol (`karol.arrrg.de`) Level erneut. Schreibe den Programmcode dieses Mal direkt in Java!
  Zur Erinnerung: Deinen Code schreibst du an der gelb markierten Stelle in der main-Methode.
- Zeige nach den Schritten jeweils das Struktogramm an und notiere auf einem **Schmierzettel**, wie du die letzte Spalte in Hefteintrag 8 befüllen würdest.
- Was passiert, wenn du dein Programm aus eigenen Methoden aufbaust? Wieso ist das so?

```
1 class Programm {
2   Robot karol = new Robot();
3
4   void main() {
5
6   }
7 }
```



Programmablauf grafisch darstellen: Struktogramme

Struktorgramme (auch:) stellen **Algorithmen (=Ablauf eines Programms)** grafisch dar. Ein Struktogramm bezieht sich immer auf genau eine Methode und stellt aufgerufene Methoden als ein Element dar.

In **Hefteintrag 8** sind die Struktogramm-Bausteine für verschiedene Befehlsarten dargestellt, die beliebig miteinander kombiniert werden können.

Zusätzlich gibt es folgenden Baustein, der eine leere Stelle (z.B. leerer else-Block) darstellt:

Programmablauf grafisch darstellen: Struktogramme

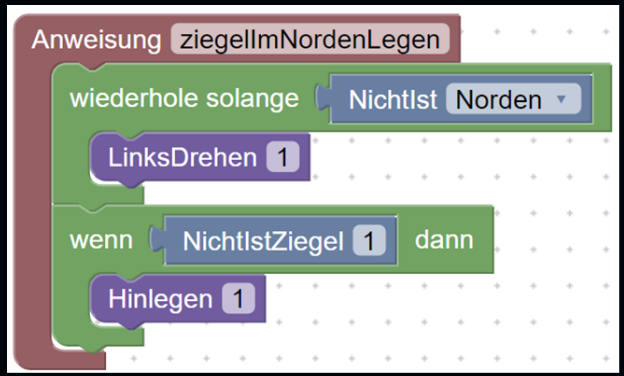
Struktorgramme (auch: `Nassi-Shneiderman-Diagramme`) stellen **Algorithmen (=Ablauf eines Programms)** grafisch dar. Ein Struktogramm bezieht sich immer auf genau eine Methode und stellt aufgerufene Methoden als ein Element dar.

In **Hefteintrag 8** sind die Struktogramm-Bausteine für verschiedene Befehlsarten dargestellt, die beliebig miteinander kombiniert werden können.

Zusätzlich gibt es folgenden Baustein, der eine leere Stelle (z.B. leerer else-Block) darstellt:

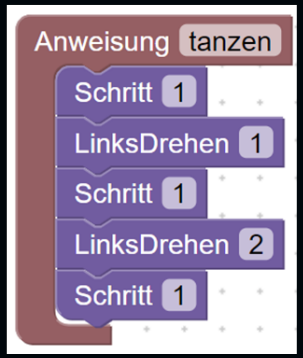
Struktogramm-Training

Zeichne zu den folgenden Code Snippets jeweils ein Struktogramm:



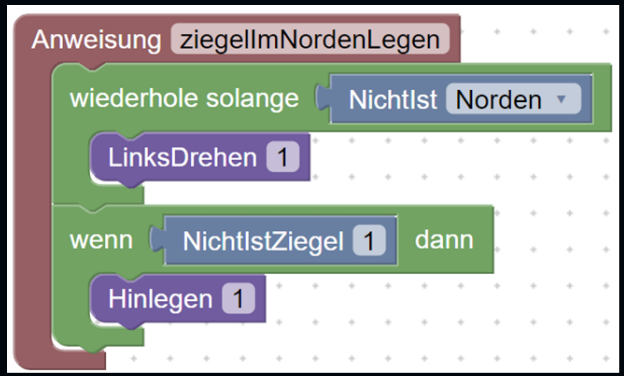

```
ziegelImNordenLegen();
quadratLaufen();
tanzen();
```

```
void quadratLaufen() {
  while (true) {
    karol.schritt(10);
    karol.linksDrehen();
  }
}
```



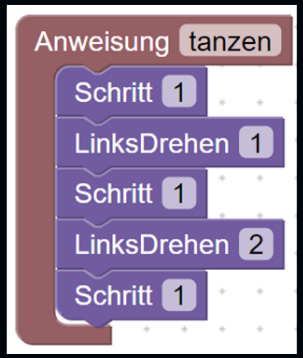
Struktogramm-Training

Zeichne zu den folgenden Code Snippets jeweils ein Struktogramm:




```
ziegelImNordenLegen();
quadratLaufen();
tanzen();
```

```
void quadratLaufen() {
  while (true) {
    karol.schritt(10);
    karol.linksDrehen();
  }
}
```



Implementiere das Klassendiagramm rechts mit Hilfe von Blockly. Probiere deine Methode aus (siehe Anleitung auf der Vorderseite). Fülle die Lücken im Java-Code unten aus, wenn deine Methoden funktionieren.

Taschenrechner
+ double addieren(double z1, double z2) + double subtrahieren(double z1, double z2) + double multiplizieren(double z1, double z2) + double dividieren(double z1, double z2) + double mittelwert(double z1, double z2)

```
public class Taschenrechner
{
    public
```

}
public

}
public

}
public

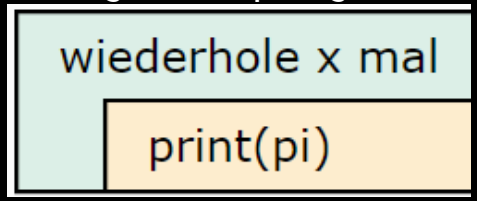
```
    }
    public


    }
}
```

Setze folgende Diagramme in BlueJ-Blockly um und notiere den Programmcode auf der nächsten Seite

Klassendiagramm:

ErweiterterTaschenrechner
+ double betrag(double z1) + void piAusgeben(double x) + double potenzieren(double b, double p)

Struktogramm für piAusgeben(double x):



Zeichne hier die Struktogramme für die restlichen Methoden:

```
public class ErweiterterTaschenrechner
{
    public
    }
    public
```

Methodenköpfe

Im können neben dem Namen noch weitere Informationen nach folgendem Schema gespeichert werden:

```
Zugriffsmodifizierer Rückgabetyp name(Datentyp parametername) { //... }
public void schritteLaufen(int anzahl, ...) { //... }
```

Methodenname und Parametername können weitgehend frei gewählt werden (Groß- und Kleinschreibung beachten!). Folgendes ist **nicht** erlaubt:

Die verwendeten haben folgende Bedeutungen:

Zugriffsmodifizierer: public - Methode kann ‚überall' verwendet werden. (im Gegensatz dazu **private**: nur in der gleichen Klasse)

Datentyp - Art der Daten (z.B. Ganzzahlen: , Kommazahlen: , Text: , Wahrheitswert:)

Rückgabetyp - Datentyp des Rückgabewerts (=). Kann z.B. Ergebnis einer Berechnung sein.

Besonderer Rückgabetyp - **void** bedeutet, die Methode hat keinen Rückgabewert. Außer bei void wird immer eine return-Anweisung benötigt.

Parameter - Speichert bei jedem Aufruf einen neuen Wert, der im Methodenrumpf über den Namen verwendet werden kann.

Methodenköpfe

Im `Methodenkopf` können neben dem Namen noch weitere Informationen nach folgendem Schema gespeichert werden:

Zugriffsmodifizierer Rückgabetyp name(Datentyp parametername) { //... }

public void schritteLaufen(int anzahl, ...) { //... }

Methodenname und Parametername können weitgehend frei gewählt werden (Groß- und Kleinschreibung beachten!). Folgendes ist **nicht** erlaubt:

Die verwendeten haben folgende Bedeutungen:

Zugriffsmodifizierer: public - Methode kann ‚überall' verwendet werden. (im Gegensatz dazu **private**: nur in der gleichen Klasse)

Datentyp - Art der Daten (z.B. Ganzzahlen: , Kommazahlen: , Text: , Wahrheitswert:)

Rückgabetyp - Datentyp des Rückgabewerts (=). Kann z.B. Ergebnis einer Berechnung sein.

Besonderer Rückgabetyp - **void** bedeutet, die Methode hat keinen Rückgabewert. Außer bei void wird immer eine return-Anweisung benötigt.

Parameter - Speichert bei jedem Aufruf einen neuen Wert, der im Methodenrumpf über den Namen verwendet werden kann.

Methodenköpfe

Im `Methodenkopf` können neben dem Namen noch weitere Informationen nach folgendem Schema gespeichert werden:

Zugriffsmodifizierer Rückgabetyp name(Datentyp parametername) { //... }

public void schritteLaufen(int anzahl, ...) { //... }

Methodenname und Parametername können weitgehend frei gewählt werden (Groß- und Kleinschreibung beachten!). Folgendes ist **nicht** erlaubt: `Leerzeichen im Namen, Zahlen am Anfang des Namens, Rechenzeichen im Namen, Schlüsselwörter als Name`

Die verwendeten haben folgende Bedeutungen:

Zugriffsmodifizierer: public - Methode kann ‚überall' verwendet werden. (im Gegensatz dazu **private**: nur in der gleichen Klasse)

Datentyp - Art der Daten (z.B. Ganzzahlen: , Kommazahlen: , Text: , Wahrheitswert:)

Rückgabetyp - Datentyp des Rückgabewerts (=). Kann z.B. Ergebnis einer Berechnung sein.

Besonderer Rückgabetyp - **void** bedeutet, die Methode hat keinen Rückgabewert. Außer bei void wird immer eine return-Anweisung benötigt.

Parameter - Speichert bei jedem Aufruf einen neuen Wert, der im Methodenrumpf über den Namen verwendet werden kann.

Methodenköpfe

Im `Methodenkopf` können neben dem Namen noch weitere Informationen nach folgendem Schema gespeichert werden:

Zugriffsmodifizierer Rückgabetyp name(Datentyp parametername) { //... }

public void schritteLaufen(int anzahl, ...) { //... }

Methodenname und Parametername können weitgehend frei gewählt werden (Groß- und Kleinschreibung beachten!). Folgendes ist **nicht** erlaubt: `Leerzeichen im Namen, Zahlen am Anfang des Namens, Rechenzeichen im Namen, Schlüsselwörter als Name`

Die verwendeten `Schlüsselwörter` haben folgende Bedeutungen:

Zugriffsmodifizierer: public - Methode kann ‚überall' verwendet werden. (im Gegensatz dazu **private**: nur in der gleichen Klasse)

Datentyp - Art der Daten (z.B. Ganzzahlen: , Kommazahlen: , Text: , Wahrheitswert:)

Rückgabetyp - Datentyp des Rückgabewerts (=). Kann z.B. Ergebnis einer Berechnung sein.

Besonderer Rückgabetyp - **void** bedeutet, die Methode hat keinen Rückgabewert. Außer bei void wird immer eine return-Anweisung benötigt.

Parameter - Speichert bei jedem Aufruf einen neuen Wert, der im Methodenrumpf über den Namen verwendet werden kann.

Methodenköpfe

Im `Methodenkopf` können neben dem Namen noch weitere Informationen nach folgendem Schema gespeichert werden:

Zugriffsmodifizierer Rückgabetyp name(Datentyp parametername) { //... }

public void schritteLaufen(int anzahl, ...) { //... }

Methodenname und Parametername können weitgehend frei gewählt werden (Groß- und Kleinschreibung beachten!). Folgendes ist **nicht** erlaubt: `Leerzeichen im Namen, Zahlen am Anfang des Namens, Rechenzeichen im Namen, Schlüsselwörter als Name`

Die verwendeten `Schlüsselwörter` haben folgende Bedeutungen:

Zugriffsmodifizierer: public - Methode kann ‚überall' verwendet werden. (im Gegensatz dazu **private**: nur in der gleichen Klasse)

Datentyp - Art der Daten (z.B. Ganzzahlen: `int`, Kommazahlen: , Text: , Wahrheitswert:)

Rückgabetyp - Datentyp des Rückgabewerts (=). Kann z.B. Ergebnis einer Berechnung sein.

Besonderer Rückgabetyp - **void** bedeutet, die Methode hat keinen Rückgabewert. Außer bei void wird immer eine return-Anweisung benötigt.

Parameter - Speichert bei jedem Aufruf einen neuen Wert, der im Methodenrumpf über den Namen verwendet werden kann.

Methodenköpfe

Im `Methodenkopf` können neben dem Namen noch weitere Informationen nach folgendem Schema gespeichert werden:

Zugriffsmodifizierer Rückgabetyp name(Datentyp parametername) { //... }

public void schritteLaufen(int anzahl, ...) { //... }

Methodenname und Parametername können weitgehend frei gewählt werden (Groß- und Kleinschreibung beachten!). Folgendes ist **nicht** erlaubt: `Leerzeichen im Namen, Zahlen am Anfang des Namens, Rechenzeichen im Namen, Schlüsselwörter als Name`

Die verwendeten `Schlüsselwörter` haben folgende Bedeutungen:

Zugriffsmodifizierer: public - Methode kann ‚überall' verwendet werden. (im Gegensatz dazu **private**: nur in der gleichen Klasse)

Datentyp - Art der Daten (z.B. Ganzzahlen: `int`, Kommazahlen: `double`, Text: , Wahrheitswert:)

Rückgabetyp - Datentyp des Rückgabewerts (=). Kann z.B. Ergebnis einer Berechnung sein.

Besonderer Rückgabetyp - **void** bedeutet, die Methode hat keinen Rückgabewert. Außer bei void wird immer eine return-Anweisung benötigt.

Parameter - Speichert bei jedem Aufruf einen neuen Wert, der im Methodenrumpf über den Namen verwendet werden kann.

Methodenköpfe

Im `Methodenkopf` können neben dem Namen noch weitere Informationen nach folgendem Schema gespeichert werden:

```
Zugriffsmodifizierer Rückgabetyp name(Datentyp parametername) { //... }
public void schritteLaufen(int anzahl, ...) { //... }
```

Methodenname und Parametername können weitgehend frei gewählt werden (Groß- und Kleinschreibung beachten!). Folgendes ist **nicht** erlaubt: `Leerzeichen im Namen, Zahlen am Anfang des Namens, Rechenzeichen im Namen, Schlüsselwörter als Name`

Die verwendeten `Schlüsselwörter` haben folgende Bedeutungen:

Zugriffsmodifizierer: public - Methode kann ‚überall' verwendet werden. (im Gegensatz dazu **private**: nur in der gleichen Klasse)

Datentyp - Art der Daten (z.B. Ganzzahlen: `int`, Kommazahlen: `double`, Text: `String`, Wahrheitswert:)

Rückgabetyp - Datentyp des Rückgabewerts (=). Kann z.B. Ergebnis einer Berechnung sein.

Besonderer Rückgabetyp - **void** bedeutet, die Methode hat keinen Rückgabewert. Außer bei void wird immer eine return-Anweisung benötigt.

Parameter - Speichert bei jedem Aufruf einen neuen Wert, der im Methodenrumpf über den Namen verwendet werden kann.

Methodenköpfe

Im `Methodenkopf` können neben dem Namen noch weitere Informationen nach folgendem Schema gespeichert werden:

Zugriffsmodifizierer Rückgabetyp name(Datentyp parametername) { //... }

public void schritteLaufen(int anzahl, ...) { //... }

Methodenname und Parametername können weitgehend frei gewählt werden (Groß- und Kleinschreibung beachten!). Folgendes ist **nicht** erlaubt: `Leerzeichen im Namen, Zahlen am Anfang des Namens, Rechenzeichen im Namen, Schlüsselwörter als Name`

Die verwendeten `Schlüsselwörter` haben folgende Bedeutungen:

Zugriffsmodifizierer: public - Methode kann ‚überall' verwendet werden. (im Gegensatz dazu **private**: nur in der gleichen Klasse)

Datentyp - Art der Daten (z.B. Ganzzahlen: `int`, Kommazahlen: `double`, Text: `String`, Wahrheitswert: `boolean`)

Rückgabetyp - Datentyp des Rückgabewerts (=). Kann z.B. Ergebnis einer Berechnung sein.

Besonderer Rückgabetyp - **void** bedeutet, die Methode hat keinen Rückgabewert. Außer bei void wird immer eine return-Anweisung benötigt.

Parameter - Speichert bei jedem Aufruf einen neuen Wert, der im Methodenrumpf über den Namen verwendet werden kann.

Methodenköpfe

Im `Methodenkopf` können neben dem Namen noch weitere Informationen nach folgendem Schema gespeichert werden:

Zugriffsmodifizierer Rückgabetyp name(Datentyp parametername) { //... }

public void schritteLaufen(int anzahl, ...) { //... }

Methodenname und Parametername können weitgehend frei gewählt werden (Groß- und Kleinschreibung beachten!). Folgendes ist **nicht** erlaubt: `Leerzeichen im Namen, Zahlen am Anfang des Namens, Rechenzeichen im Namen, Schlüsselwörter als Name`

Die verwendeten `Schlüsselwörter` haben folgende Bedeutungen:

Zugriffsmodifizierer: public - Methode kann ‚überall' verwendet werden. (im Gegensatz dazu **private**: nur in der gleichen Klasse)

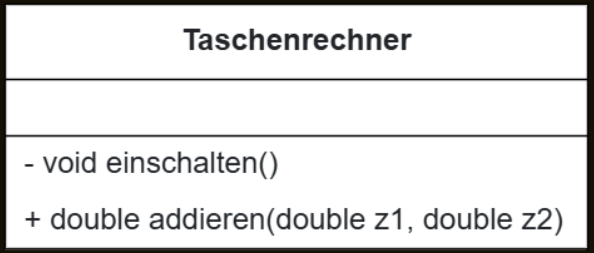
Datentyp - Art der Daten (z.B. Ganzzahlen: `int`, Kommazahlen: `double`, Text: `String`, Wahrheitswert: `boolean`)

Rückgabetyp - Datentyp des Rückgabewerts (= `return-value`). Kann z.B. Ergebnis einer Berechnung sein.

Besonderer Rückgabetyp - **void** bedeutet, die Methode hat keinen Rückgabewert. Außer bei void wird immer eine return-Anweisung benötigt.

Parameter - Speichert bei jedem Aufruf einen neuen Wert, der im Methodenrumpf über den Namen verwendet werden kann.

Methoden im Klassendiagramm




```
public class Taschenrechner {

    private void einschalten() {

        // ...

    }

    public double addieren(double z1,double z2) {

        return z1 + z2;

    }

}
```

Attribute im Klassendiagramm

Findet heraus, wie Attrbiute im Klassendiagramm dargestellt werden. Achtet auf **Zugriffsmodifizierer, Datentyp und Name**. Macht eine Skizze auf einem Schmierblatt.

Attribute im Klassendiagramm

Attribute werden im Klassendiagramm folgendermaßen dargestellt:

Taschenrechner

Struktogramme und Klassendiagramme